

ADK Master Guide

The complete guide to building DataByt's AI support agent — from understanding what you're building, to the Python you need, to every line of code explained, to a live agent in production.

This guide teaches you to OWN the code, not just run it.

Every code block has a line-by-line breakdown telling you what it does, why it's written that way, and what breaks if you change it. After this guide you can extend, debug, and modify the agent yourself.

Python · Google ADK · Gemini Flash · Supabase · adk web · Cloud Run · Next.js | June 2026

CHAPTER 1

The Aim — What Are We Building and Why?

The Problem (in one sentence)

A CFO paying ₹39,000/month expects an answer within minutes. You're one person. You can't be awake 24/7.

The Solution

An AI support agent that understands DataByt deeply — not a generic chatbot, but a specialist that knows your product, your customers, and can take real actions in your system.

What the finished agent does:

Customer types a question in DataByt dashboard

↓

Agent reads it and decides: "what kind of question is this?"

↓

HOW DO I...?	→ gives the exact steps
WHAT IS STATUS OF...?	→ checks your database
I WANT REFUND / BUG	→ flags you with context

↓

Customer gets a reply in under 30 seconds

You get notified only for the hard 30% that needs you

The "Done" Picture

You'll know you're done when this works:

Customer asks: "How do I connect QuickBooks?"
Agent replies: "Go to Dashboard → Integrations → Connect QuickBooks.
Log in with your QuickBooks account and click Allow.
DataByt will import your overdue invoices automatically.
Setup takes about 5 minutes."

Customer asks: "How many invoices are overdue?"
Agent replies: "You currently have 5 overdue invoices
totalling ₹4,20,000." ← real number from your database

Customer says: "I want a refund."
Agent replies: "I understand. I've flagged this for our team
and someone will follow up with you shortly."
Agent sends YOU: "Refund request from org abc-123. Ticket ID: t-991"

Why Python + Google ADK? ADK (Agent Development Kit) is Google's open-source framework for building AI agents. It handles the plumbing — routing messages, calling functions, managing conversation memory — so you write only the business logic. Python because ADK is most mature in Python, and because the [adk web](#) visual UI (which you'll use to debug) is Python-only. You'll also use Python at work if you ever join an AI team.

The Plan — 7 Milestones to a Live Agent

You build in 7 milestones. Each milestone ends with something you can see working. Don't skip — each one builds on the last.

#	Milestone	What you build	What you'll see	Time
0	Setup	Python, ADK, Gemini key, folder	<code>adk --version</code> prints a number	15 min
1	Hello Agent	The simplest agent	It talks to you in a browser	20 min
2	Agent + Tool	Agent that calls a function	It says "5 overdue" from a fake count	30 min
3	Knowledge Agent	Specialist that answers how-to questions	Correct answer about QuickBooks, honest "I don't know" otherwise	30 min
4	The Manager	Multi-agent routing system	3 different questions go to 3 different specialists	45 min
5	Real Data	Tool that reads your Supabase database	Agent reports the REAL invoice count from your database	45 min
6	Deploy	Agent live on the internet	A public URL that returns agent replies	45 min
7	Wire In	DataByt calls the agent	<code>/api/support</code> on your live site answers	30 min

Rule for every milestone: do not move to the next one until the checkpoint for this one passes. If you move forward with a broken foundation, the next milestone will break in a confusing way and you won't know why.

The Tools — What Each One Does and Why It's Here

Tool	What it is	Why we use it	Where in the build
Python 3.11	Programming language	ADK runs on Python; most AI libraries are Python-first	Every file you write
Google ADK	Agent framework — the scaffolding that makes agents work	Handles routing, tool-calling, conversation memory. Without it you'd write all this yourself.	Every agent file
Gemini 2.0 Flash	The AI brain (LLM)	Reads customer questions and decides what to do. Flash = fast + cheap. It's what makes the agent "intelligent".	Inside every Agent()
adk web	Visual development UI	A browser-based dashboard that shows every decision the agent makes in real time. Your primary debugging tool.	Milestones 1–5
pip	Python package installer	How you install ADK, Supabase client, and other packages	Milestone 0
venv	Virtual environment	A clean sandbox so packages don't conflict with other projects	Milestone 0
Supabase Python	Database client library	Lets Python read from your Supabase database directly	Milestone 5
Google Cloud CLI (gcloud)	Command-line tool for Google Cloud	Deploys your agent to Cloud Run with one command	Milestone 6
Cloud Run	Serverless hosting on Google Cloud	Runs your Python agent on a public URL. "Scales to zero" = costs nothing when idle.	Milestone 6
PowerShell	Your command-line shell on Windows	Where you type commands to install, run, and deploy	Throughout
Next.js (TypeScript)	Your existing DataByt web app	The front-end that will call the agent via one API route	Milestone 7

What you DON'T need: Docker, Kubernetes, databases beyond Supabase, separate servers, Python web frameworks (Flask/FastAPI), LangChain, LangGraph. ADK handles all the plumbing. Keep it simple.

The Python You Need — Only This, Nothing More

3 concepts. Each explained with a DataByt example. Master these 3, build the whole agent.

This is NOT a Python course. This is the exact Python used in this build — no more. You don't need loops, classes, inheritance, file I/O, list comprehensions, or anything else that's not listed here.

1. Variables — storing a value

`agent.py`

```
status = "overdue"      # stores the text "overdue" and names it "status"
count  = 5               # stores the number 5 and names it "count"
url    = "https://..." # stores a URL
```

WHAT THIS TEACHES YOU

<code>=</code> (single equals)	means "store this value". It's NOT checking equality (<code>==</code> does that).
No type declaration	Python figures out the type automatically. <code>status = "overdue"</code> is already a string — no need to say <code>String status</code> .
Why it matters	You'll store your Supabase client, your API responses, and your env vars in variables.

2. Dictionaries — the most important data structure in ADK

Every tool you build MUST return a dictionary. The agent reads dictionaries to understand what a tool found. This is the number one thing to understand.

`agent.py`

```
# A dictionary is a labeled container — like a form with filled-in fields
result = {"status": "success", "count": 5}

# Reading a value from a dictionary by its label
print(result["count"])  # → prints: 5
print(result["status"]) # → prints: success

# The two dictionaries ALL your tools must return:
# When it worked:
return {"status": "success", "count": 5}
# When it failed:
return {"status": "error", "message": "Could not reach database."}
```

WHAT THIS TEACHES YOU

<code>{ }</code> curly braces	create a dictionary
<code>"key": value</code>	a label (string) followed by a colon, then the value. Pairs separated by commas.
<code>result["count"]</code>	reads the value stored at label "count"
Why "status" key	ADK reads the status key to know if the tool succeeded. Always include it.

Why dictionaries? When your tool runs, the agent receives its return value. The agent reads the dictionary to form its reply. "count: 5" becomes "You have 5 overdue invoices." The agent can't work with raw numbers — it needs labeled data.

3. Functions — reusable named blocks of code

agent.py

```
def get_invoice_count(status):
    # everything indented here is inside the function
    count = 5          # pretend we looked this up
    return count        # sends the result back to whoever called it

# Calling the function:
result = get_invoice_count("overdue")  # result is now 5
```

LINE BY LINE

def	short for "define". Tells Python you're creating a function.
get_invoice_count	the function's name. Use lowercase with underscores. Verbs preferred.
(status)	the parameter — a variable that gets its value when someone calls the function. You can have multiple: (status, org_id)
: (colon)	starts the function body. Everything indented below this belongs to the function.
indentation (4 spaces)	Python uses indentation (not curly braces) to group code. Wrong indentation = error.
return	sends a value back to whoever called the function and exits it. Without return, the function returns None.

4. Type hints + docstrings — critical for ADK tools

Type hints and docstrings are how ADK tells Gemini what your tool does and what to pass to it. This is the most ADK-specific thing to understand.

agent.py

```
def get_invoice_count(status: str) -> dict:
    """Returns how many invoices have a given status.

    Args:
        status: One of 'open', 'overdue', or 'paid'.

    Returns:
        A dict with the count of matching invoices.
    """
    fake = {"open": 12, "overdue": 5, "paid": 240}
    return {"status": "success", "count": fake.get(status, 0)}
```

LINE BY LINE — EVERY PART MATTERS

status: str	type hint — tells ADK this parameter is a string (text). ADK passes this to Gemini so Gemini knows what to send.
-> dict	return type hint — says this function returns a dictionary. The arrow is just syntax.
"""..."""	docstring — a special string that describes what the function does. ADK gives this to Gemini. Gemini reads it to decide WHEN to call this tool and what to pass. A vague docstring = Gemini misuses the tool.
Args: section	documents each parameter. Gemini reads this to know what values are valid for each argument.
fake.get(status, 0)	looks up "status" in the fake dictionary. If it's not there, returns 0 instead of crashing.

The docstring is NOT for you — it's for Gemini. When a customer asks "how many invoices are overdue?", Gemini reads all your tool docstrings and picks the right one. If your docstring says "does stuff with invoices" instead of "returns the count of invoices with a given status", Gemini won't know to use it — or will use it incorrectly.

5. f-strings — inserting variables into text

agent.py

```
org = "Dinometrics"
count = 5

# f-string: put f before the quote, use {} to insert variables
message = f"Hello {org}. You have {count} overdue invoices."
# result: "Hello Dinometrics. You have 5 overdue invoices."

# Also useful in error messages:
return {"status": "error", "message": f"Unknown status: {status}"}
```

WHAT THIS TEACHES YOU

<code>f"..."</code>	the f before the quote makes it an f-string. Without f, {} is just text.
<code>{variable}</code>	Python replaces this with the variable's value when the string is created.

6. try/except — handling errors without crashing

Every tool that talks to a database or API must use try/except. If the database is down and your tool crashes (raises an exception), the entire agent dies. Instead, catch the error and return it as a dictionary.

agent.py

```
def get_invoice_count(status: str, org_id: str) -> dict:
    """..."""
    try:
        # code that might fail (network call, database query)
        result = database.query(status, org_id)
        return {"status": "success", "count": result.count}
    except Exception as e:
        # if ANYTHING goes wrong, we end up here instead of crashing
        return {"status": "error", "message": str(e)}
```

LINE BY LINE

<code>try:</code>	run the indented code; watch for errors
<code>except Exception as e:</code>	if ANY error occurs, catch it and call it "e". Exception = any kind of error.
<code>str(e)</code>	converts the error object to a readable string. Eg: "Connection refused"
<code>why not crash</code>	if your tool raises an exception, ADK can't process it. The agent returns nothing. The customer sees a blank reply. Always catch and return an error dict.

7. import — using code from other packages

agent.py

```
from google.adk.agents import Agent          # get the Agent class from ADK
from google.adk.agents import SequentialAgent # and this one too
import os                                    # the os module (for env vars)
from supabase import create_client           # the Supabase client function
```

TWO FORMS OF IMPORT

<code>from X import Y</code>	get specifically Y from package X. Use this when you only need specific things.
<code>import X</code>	import the whole module X. Access things with <code>X.something</code> . eg: <code>os.environ</code>

8. Environment variables — secrets that don't go in the code

`.env file`

```
GOOGLE_API_KEY=AiZaSy...
SUPABASE_URL=https://yourproject.supabase.co
SUPABASE_SERVICE_KEY=eyJhbGc...
```

`agent.py`

```
import os

api_key = os.environ["GOOGLE_API_KEY"]    # reads the value from .env
db_url  = os.environ["SUPABASE_URL"]      # reads another value

# ADK reads the .env file automatically when you run adk web.
# You don't need to do anything special — just have the .env in the agent folder.
```

WHY ENV VARS EXIST

Security

API keys must never be written directly in code. If you commit code to git, the key becomes public. .env files are in .gitignore.

`os.environ["KEY"]`

raises a `KeyError` if the variable doesn't exist. Use this so you know immediately when a key is missing.

9. Using a class without writing one

Agent, SequentialAgent, and other ADK objects are *classes*. You use them by creating *instances*. You do not write the class yourself.

`agent.py`

```
# Agent is a class that Google wrote inside the ADK package.
# You create an "instance" of it by calling Agent() with parentheses
# and filling in the parameters.

my_agent = Agent(
    name="support_agent",      # a parameter named "name"
    model="gemini-2.0-flash",  # a parameter named "model"
    instruction="...",         # a parameter named "instruction"
)

# my_agent is now a fully configured agent object.
# You can pass it to sub_agents, or name it root_agent for ADK to run it.
```

WHAT THIS TEACHES YOU

`Agent(...)`

creates a new instance. Every time you call `Agent()` you get a fresh, separate agent.

`name= model= instruction=`

these are keyword arguments. Order doesn't matter; the name matters.

You don't write `class Agent:`

Google wrote that. Your job is to configure and combine instances.

That's all the Python you need. Variables, dictionaries, functions, type hints, docstrings, f-strings, try/except, imports, env vars, and using a class. If you understand these 9 things, you can write and understand every line of code in this build.

The Build — 7 Milestones With Every Line Explained

0

Setup — tools installed and working

~15 minutes · do this once, never again

0.1 — Verify Python

PowerShell

```
python --version
```

WHAT THIS COMMAND DOES

<code>python</code>	runs the Python program
<code>--version</code>	tells it to print its version number and exit
You should see	Python 3.11.x (any 3.10 or higher is fine)

🔧 **"python is not recognized"** → Download Python from python.org/downloads. During install, **tick the checkbox "Add Python to PATH"**. Close PowerShell, open a new one, try again.

0.2 — Create the agents folder and virtual environment

PowerShell

```
cd d:\AI\DP\databyt-saas      # navigate to your DataByt project
mkdir agents                  # create a new folder called agents
cd agents                     # enter it
python -m venv .venv          # create a virtual environment named .venv
.venv\Scripts\Activate.ps1    # activate it — your prompt shows (.venv)
```

WHAT EACH LINE DOES

<code>mkdir agents</code>	makes a new directory. The Python agent lives here, separate from the TypeScript DataByt code.
<code>python -m venv .venv</code>	<code>-m</code> means "run this module". <code>venv</code> creates a virtual environment. <code>.venv</code> is just the folder name (convention).
<code>Activate.ps1</code>	a PowerShell script that makes this terminal session use the <code>venv</code> 's Python. Your prompt changes to <code>(.venv)</code> to confirm it's active.
Why <code>venv</code>	without it, <code>pip</code> installs packages globally and different projects can have version conflicts. <code>venv</code> isolates them.

🔧 **"running scripts is disabled"** → run once: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` then activate again.

0.3 — Install ADK

PowerShell

```
pip install google-adk      # installs ADK and all its dependencies
```

WHAT THIS DOES

<code>pip</code>	Python's package installer. It downloads packages from the internet.
<code>google-adk</code>	the package name. pip finds it on pypi.org (Python's package registry).
You should see	many lines ending in "Successfully installed google-adk-1.x.x ..."

0.4 — Get your free Gemini API key

1. Open aistudio.google.com/apikey · 2. Sign in · 3. Click "Create API key" → "Create in new project" · 4. Copy the key (starts with `AIzaSy`).

✓ CHECKPOINT 0 — Run this:

PowerShell

```
adk --version
```

You should see: a version number like `1.x.x`

🔧 **"adk not recognized"** → your venv is not active. Run `.venv\Scripts\Activate.ps1` (prompt must show `(.venv)`), try again.

1

Hello Agent — the simplest agent, talking in a browser

~20 minutes · proves the whole pipeline works before adding complexity

1.1 — Create the agent folder structure

PowerShell (from agents/)

```
mkdir support_agent # create a folder for this agent
cd support_agent    # enter it
```

Now create three files inside `support_agent/`. Use any text editor (Notepad, VS Code).

File 1 of 3 — `__init__.py`

`support_agent/__init__.py`

```
from . import agent
```

WHAT THIS FILE DOES — EVERY CHARACTER MATTERS

`__init__.py`

a special Python file that marks this folder as a "package". Without it, ADK can't find your agent. It must be named exactly `__init__.py`.

`from . import agent`

the dot (.) means "from this same folder". This imports your `agent.py` file when ADK loads the package.

File 2 of 3 — `.env`

`support_agent/.env`

```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=AIZA_sY_paste_your_real_key_here
```

WHAT EACH LINE DOES

`GOOGLE_GENAI_USE_VERTEXAI=FALSE`

tells ADK to use the free Gemini API (aistudio.google.com), NOT Google Cloud's Vertex AI. Keep this FALSE until you deploy. Never add spaces around the =.

`GOOGLE_API_KEY=AIZA...`

your Gemini API key. ADK reads this automatically when it starts.

`no quotes around values`

unlike some other systems, `.env` files don't use quotes. Just `KEY=value`.

File 3 of 3 — `agent.py`

`support_agent/agent.py`

```
from google.adk.agents import Agent

root_agent = Agent(
    name="support_agent",
    model="gemini-2.0-flash",
    description="DataByt customer support assistant.",
    instruction="""
You are DataByt's friendly support assistant.
DataByt automates accounts-receivable (AR) collections for B2B finance teams.
Answer questions in a clear, professional, and concise way.
If you don't know something, say so honestly — never make up facts.
""",
)
```

EVERY LINE EXPLAINED

<code>from google.adk.agents import Agent</code>	brings the Agent class from ADK into this file. Without this, Python doesn't know what "Agent" is.
<code>root_agent = Agent(...)</code>	root_agent is the exact variable name ADK looks for. If you name it my_agent or anything else, the adk web dropdown will be empty. This is a hard requirement.
<code>name="support_agent"</code>	appears in the Events panel and in the /run API call. Can be any string but must be consistent. Use lowercase-with-underscores.
<code>model="gemini-2.0-flash"</code>	which AI brain to use. Flash is fast and cheap — costs ~\$0.075 per 1M tokens. Use Flash unless you have a specific reason to upgrade. "Pro" models cost 10-30x more.
<code>description="..."</code>	used by a PARENT agent to decide when to route to this one. Even for the root agent, it's good practice to include it.
<code>instruction="""..."""</code>	the agent's job description. Gemini reads this before every response. 90% of agent quality comes from writing this well. Triple-quotes (""") allow multi-line strings.

1.2 — Run the visual UI

PowerShell (from agents/, NOT from inside support_agent)

```
cd ..           # go UP to the agents folder (the parent of support_agent)
adk web         # launches the visual development server
```

WHY YOU RUN FROM THE PARENT FOLDER

<code>adk web</code> looks for agent folders inside the current directory	If you run it from inside support_agent, it looks for sub-folders there and finds nothing. Always run from the folder that CONTAINS your agent folders.
You should see	INFO: Uvicorn running on http://127.0.0.1:8000

✔ CHECKPOINT 1

Open <http://localhost:8000> · Select `support_agent` from the dropdown · Type: `Hi, what is DataByt?` · Press Enter

You should see: the agent replies with a description of DataByt.

Now explore the UI — this is your most important debugging tool:

- **Events panel:** every step the agent took to generate the reply. You'll watch tool calls happen here.
- **State panel:** the agent's memory for this conversation (empty for now).
- **Trace:** how long each step took.

- 🔧 **Dropdown is empty** → you ran `adk web` from the wrong folder. Ctrl-C, confirm you're in `agents` (not inside support_agent), run again.
- 🔧 **API error / 403** → wrong API key in .env. Re-copy from aistudio.google.com/apikey. No spaces, no quotes.

An agent with no tools can only chat. A tool lets it call a function and use the result. We give the agent one tool that "looks up" invoice counts (fake data for now).

Stop the server (Ctrl-C). Replace the entire contents of `support_agent/agent.py`:

support_agent/agent.py — full file

```
from google.adk.agents import Agent          # same as before

# -----
# THE TOOL — a plain Python function
# ADK turns this into something Gemini can call
# -----
def get_invoice_count(status: str) -> dict:
    """Returns how many invoices have a given status.

    Args:
        status: One of 'open', 'overdue', or 'paid'.

    Returns:
        A dict with a 'count' key containing the number of invoices.
    """
    fake_database = {"open": 12, "overdue": 5, "paid": 240}

    if status not in fake_database:
        return {"status": "error", "message": f"Unknown status: {status}"}

    return {"status": "success", "count": fake_database[status]}

# -----
# THE AGENT — now with a tool
# -----
root_agent = Agent(
    name="support_agent",
    model="gemini-2.0-flash",
    description="DataByt support assistant that can check invoice counts.",
    instruction="""
    You are DataByt's support assistant.
    When asked how many invoices are open, overdue, or paid,
    ALWAYS use the get_invoice_count tool to find the number.
    Never guess a number. Report what the tool returns.
    """,
    tools=[get_invoice_count],    # ← this line gives the agent the tool
)
```

EVERY NEW LINE EXPLAINED

<code>def get_invoice_count(status: str) -> dict:</code>	defines the tool. It's a plain Python function — no special decorator or ADK class needed. ADK detects it's a tool when you add it to the <code>tools</code> list.
<code>the docstring ("""...""")</code>	Gemini reads this to know what the function does and when to call it. The Args section tells Gemini what values to pass for each parameter.
<code>fake_database = {...}</code>	a dictionary holding pretend data. Key = status string, Value = count. We replace this with a real database call in Milestone 5.
<code>if status not in fake_database:</code>	checks that the status is one we know. If Gemini passes "cancelled" (which we don't handle), we return an error instead of crashing.
<code>tools=[get_invoice_count]</code>	a list of functions the agent can call. The square brackets <code>[]</code> create a list. You can add more tools here: <code>tools=[tool1, tool2, tool3]</code> . The function name goes WITHOUT parentheses — you're passing the function itself, not calling it.

✓ CHECKPOINT 2

Run `adk web` from `agents/`. Ask: `How many invoices are overdue?`

You should see: the agent replies with "5".

More importantly: click the **Events** panel. You should see an entry showing:

→ Agent called `get_invoice_count` with argument `status: "overdue"`

→ Tool returned `{"status": "success", "count": 5}`

→ Agent then wrote its reply using that number

This is how you debug: if the agent says the wrong number, look at what the tool returned in Events. If the tool returned correctly but the agent said something wrong, the instruction is the problem.

3

Knowledge Agent — answers "how do I..." questions from its own knowledge

~30 minutes · the how-to specialist, honest when it doesn't know

Create a second, separate agent. It's a specialist for product questions. Ctrl-C to stop the server.

PowerShell (from `agents/`)

```
mkdir knowledge_agent
copy support_agent\.env knowledge_agent\.env # reuse the same API key
```

`knowledge_agent/__init__.py`

```
from . import agent
```

`knowledge_agent/agent.py` — full file

```

from google.adk.agents import Agent

root_agent = Agent(
    name="knowledge_agent",
    model="gemini-2.0-flash",
    description="Answers how-to and product questions about DataByt.",
    instruction="""
You are DataByt's product knowledge specialist.
Answer ONLY using the knowledge section below.
If a question is not covered in the knowledge section, say:
"I'm not sure about that. Let me connect you with our team."
Never invent steps or information not listed here.

— KNOWLEDGE —

CONNECTING QUICKBOOKS / XERO:
Go to Dashboard → Integrations. Click "Connect QuickBooks"
(or Xero). Log in to your accounting account and click Allow.
DataByt then imports your overdue invoices daily. Takes 5 minutes.

DUNNING EMAILS:
DataByt sends one email per customer covering all their overdue
invoices. Tone escalates: L1 (friendly, 1+ days late),
L2 (firm, 10+ days), L3 (final notice, 30+ days).
Change day thresholds in Settings → Collection Rules.

PAYMENT LINKS:
Set your own payment link in Settings → Payment Collection.
Works with Stripe, Razorpay, bank transfer, or any URL.

DISPUTES:
File a dispute from the AR Aging page. Collections pause for that
customer. Investigate, then resolve or reject. Collections resume.

REPORTS:
Dashboard → Reports → "Generate PDF Report" for a board-ready PDF.

PRICING:
One plan, everything included. Founding rate locked for first 20
customers. 30-day free trial, no credit card needed.

— END KNOWLEDGE —
""",
)

```

WHY THIS DESIGN WORKS

instruction contains the knowledge	for v1, the knowledge lives directly in the instruction. Simple, works immediately, no extra setup. Later you can upgrade to real RAG (vector search over your docs).
"ONLY using the knowledge section"	Gemini by default will answer anything from its training data. This line constrains it to your knowledge only.
the "I'm not sure" line	tells it what to say when the answer isn't in the knowledge. Without this, it might hallucinate a convincing but wrong answer.
— markers —	visual separators in the instruction help the model distinguish knowledge from instructions. Not required, but improves accuracy.

✓ CHECKPOINT 3

adk web · pick knowledge_agent

Ask: How do I connect QuickBooks? → gives the 5-minute Integrations steps ✓

Ask: What is the capital of France? → says it's not sure / offers the team ✓

The "I don't know" is correct. That means it won't lie to your customers about things outside DataByt.

4

The Manager — multi-agent routing

~45 minutes · the heart of the system

Now we build the manager that reads each ticket and routes it to the right specialist. This is the single most important pattern in ADK. Ctrl-C to stop the server. Replace the entire contents of `support_agent/agent.py` :

```
support_agent/agent.py - full file
```



```

from google.adk.agents import Agent

# — TOOL (still fake data, replaced in Milestone 5) —————

def get_invoice_count(status: str) -> dict:
    """Returns how many invoices have a given status.

    Args:
        status: One of 'open', 'overdue', or 'paid'.
    """
    fake = {"open": 12, "overdue": 5, "paid": 240}
    return {"status": "success", "count": fake.get(status, 0)}

# — SPECIALIST 1: knowledge (how-to questions) —————

knowledge_agent = Agent(
    name="knowledge_agent",
    model="gemini-2.0-flash",
    description="Answers how-to, product, pricing, and feature questions about DataByt.",
    instruction="""
    You answer product questions about DataByt using ONLY this knowledge.
    If the answer is not here, say "I'm not sure – let me connect you with our team."

    - CONNECT QB/XERO: Dashboard → Integrations → Connect, log in, Allow. 5 min.
    - DUNNING: one email/customer, L1/L2/L3 tone by days overdue. Settings → Collection Rules.
    - PAYMENT LINKS: Settings → Payment Collection. Any payment URL.
    - DISPUTES: file on AR Aging page; collections pause; resolve to resume.
    - REPORTS: Dashboard → Reports → Generate PDF Report.
    - PRICING: one plan, founding rate for first 20, 30-day free trial.
    """,
)

# — SPECIALIST 2: account (data questions) —————

account_agent = Agent(
    name="account_agent",
    model="gemini-2.0-flash",
    description="Checks the customer's invoice counts and account status data.",
    instruction="""
    You look up real account data for the customer.
    When asked how many invoices are open, overdue, or paid,
    ALWAYS call the get_invoice_count tool.
    Never guess numbers. Report what the tool returns clearly.
    """,
    tools=[get_invoice_count],
)

# — SPECIALIST 3: escalation (hard tickets) —————

escalation_agent = Agent(
    name="escalation_agent",
    model="gemini-2.0-flash",
    description="Handles refunds, billing disputes, bugs, and upset or frustrated customers.",
    instruction="""
    You handle the difficult tickets that need human attention.
    Apologise briefly and professionally.
    Tell the customer that a team member will follow up within a few hours.
    Ask if there's any specific detail that would help the team assist them faster.
    Do not promise outcomes (eg. "we will refund you") – just acknowledge and escalate.
    """,
)

# — THE MANAGER (root_agent) —————
# This is what ADK runs. It reads the ticket and routes it.
#

root_agent = Agent(
    name="support_manager",
    model="gemini-2.0-flash",
    description="DataByt front-line support manager.",

```

```

instruction="""
You are DataByt's support manager. Your only job is to read
the customer's message and route it to the right specialist.

Routing rules:
- "How do I...", product features, pricing, setup questions
  → transfer to knowledge_agent

- "How many invoices...", "What is the status of...",
  any question about their specific data
  → transfer to account_agent

- Refunds, billing disputes, complaints, bugs,
  angry language, or anything you are not sure about
  → transfer to escalation_agent

IMPORTANT: Always route. Never answer a specialist question yourself.
You are the router, not the responder.
""",
sub_agents=[knowledge_agent, account_agent, escalation_agent],
)

```

THE KEY NEW LINES EXPLAINED

the three <code>Agent()</code> blocks above <code>root_agent</code>	these are the specialists. They are defined as variables (<code>knowledge_agent</code> , <code>account_agent</code> , <code>escalation_agent</code>) but they are NOT named <code>root_agent</code> . ADK only runs <code>root_agent</code> directly — the others are only reached through routing.
description on each specialist	the manager uses the description to decide which specialist to route to. Make each description clearly state what kind of tickets it handles. This is what Gemini reads when deciding.
<code>sub_agents=[knowledge_agent, account_agent, escalation_agent]</code>	registers the three specialists as children of the manager. The order doesn't matter — the manager decides routing from the descriptions and its instruction.
"Always route. Never answer a specialist question yourself."	without this line, the manager might answer product questions itself (partially). This forces it to always delegate.

Why this structure? One agent with one giant instruction that handles all cases fails — it gets confused, contradicts itself, and gives inconsistent answers. Specialists with narrow, clear instructions stay focused. The manager's only job is routing — it does one thing well.

✅ CHECKPOINT 4

`adk web` · pick `support_agent` (now the manager) · test all three paths and watch Events each time:

1. `How do I connect QuickBooks?` → Events shows transfer to **knowledge_agent** → gives steps ✅
2. `How many invoices are overdue?` → transfer to **account_agent** → calls tool → says "5" ✅
3. `I want a refund` → transfer to **escalation_agent** → apologises, promises follow-up ✅

If all three route correctly, you've built a real multi-agent system.

- 🔧 **Everything goes to one specialist** → the descriptions are too similar. Make each description explicitly state different trigger words. Look at what message went wrong in Events → check which description matched it → rewrite that description to not match incorrectly.
- 🔧 **Manager answers instead of routing** → add "Always route. Never answer a specialist question yourself." to the manager's instruction.

5

Real Data — read actual invoices from Supabase

~45 minutes · replaces fake_database with your real database

Ctrl-C. Install the Supabase Python client:

PowerShell (venv must be active)

```
pip install supabase
```

Add your database credentials to .envOpen `support_agent/.env` and add two lines. Get these from your Vercel environment variables or from Supabase → Project Settings → API:**support_agent/.env — add these two lines**

```
SUPABASE_URL=https://yourprojectref.supabase.co
SUPABASE_SERVICE_KEY=eyJhbGciOiJIUzI1NiIs... # use the service_role key, NOT the anon key
```

service_role key vs anon key: the anon key has Row Level Security (RLS) applied — the agent can't read other orgs' data. The service_role key bypasses RLS so the agent can read any org's data. Use service_role only in the backend (never in browser code). Keep .env out of git.

Replace get_invoice_count with the real versionIn `support_agent/agent.py`, replace ONLY the `get_invoice_count` function. Leave everything else unchanged:**support_agent/agent.py — only this function changes**

```
import os
from supabase import create_client

# create the Supabase client ONCE when the module loads
# (not inside the function, so we don't reconnect on every call)
_supabase = create_client(
    os.environ["SUPABASE_URL"],
    os.environ["SUPABASE_SERVICE_KEY"],
)

def get_invoice_count(status: str, org_id: str) -> dict:
    """Returns how many invoices an organisation has with a given status.

    Args:
        status: One of 'open', 'overdue', or 'paid'.
        org_id: The organisation's unique ID (UUID string).
    """
    try:
        response = (
            _supabase
            .table("invoices")           # which table to query
            .select("id", count="exact") # select id column, get exact count
            .eq("org_id", org_id)        # filter: org_id must match
            .eq("status", status)        # filter: status must match
            .execute()                   # run the query
        )
        return {"status": "success", "count": response.count or 0}
    except Exception as e:
        return {"status": "error", "message": str(e)}
```

EVERY NEW LINE EXPLAINED

<code>_supabase = create_client(...)</code>	creates a connection to your Supabase database. The leading underscore <code>_</code> is a Python convention meaning "this is a module-level private variable". We create it once here (not inside the function) so it doesn't reconnect on every call.
<code>os.environ["SUPABASE_URL"]</code>	reads the value from the <code>.env</code> file. Raises <code>KeyError</code> immediately if the variable is missing — better than silently getting <code>None</code> .
<code>.table("invoices")</code>	picks which database table to query — like <code>FROM invoices</code> in SQL.
<code>.select("id", count="exact")</code>	we don't need the actual data, just the count. <code>count="exact"</code> tells Supabase to return the total row count in <code>response.count</code> .
<code>.eq("org_id", org_id)</code>	adds a filter: only rows where <code>org_id</code> column equals the <code>org_id</code> parameter. Like <code>WHERE org_id = '...'</code> in SQL.
<code>.eq("status", status)</code>	second filter: only rows where <code>status</code> column equals the <code>status</code> parameter.
<code>.execute()</code>	actually runs the query. Without this the query is just built, not sent.
<code>response.count or 0</code>	<code>response.count</code> is the row count. "or 0" means: if count is <code>None</code> (no rows found), use 0 instead.
<code>try/except Exception as e</code>	if the database is unreachable, the query fails, or any other error occurs, we catch it and return a proper error dictionary instead of crashing the agent.

Also update `account_agent`'s instruction to ask for `org_id` (since the tool now needs it):

account_agent instruction — update this line

```
instruction="""
You look up real account data.
When asked about invoice counts, ALWAYS call get_invoice_count
with the status AND the org_id.
If you don't know the org_id, ask the user for it.
Never guess numbers — always call the tool.
"""
```

✓ CHECKPOINT 5

`adk web` → pick `support_agent`

Ask: How many overdue invoices does org YOUR-REAL-ORG-ID have?

(Find a real `org_id` in your Supabase `organizations` table)

You should see the REAL count from your database — not 5.

In Events, you'll see the tool was called with your `org_id` and `status`, and returned the real count.

🔧 **KeyError: SUPABASE_URL** → the line is missing from `.env`, or there's a typo. The key must be exactly `SUPABASE_URL`.

🔧 **Auth error / JWT invalid** → you used the anon key. Use the `service_role` key (from Supabase → Project Settings → API → `service_role` section).

🔧 **count is 0 but you expect more** → try a different status ('open', 'paid') or verify the `org_id` is correct in your `organizations` table.

6.1 — Install Google Cloud CLI

Download and install from cloud.google.com/sdk/docs/install (Windows installer). Accept all defaults. Close and reopen PowerShell. Reactivate your venv:

PowerShell

```
cd d:\AI\DP\databyt-saas\agents
.venv\Scripts\Activate.ps1
```

6.2 — Authenticate and configure

PowerShell

```
gcloud auth login           # opens browser, sign in with your Google account
gcloud projects list        # shows your projects — note the PROJECT_ID
gcloud config set project YOUR_PROJECT_ID # set active project
gcloud services enable run.googleapis.com aiplatform.googleapis.com # turn on services
```

WHAT EACH LINE DOES

<code>gcloud auth login</code>	logs gcloud into your Google account, which owns the Cloud project and credits
<code>gcloud services enable ...</code>	Cloud Run and Vertex AI are "services" that must be explicitly enabled. This is a one-time step per project.

6.3 — Create requirements.txt

Cloud Run needs a file listing your dependencies. Create `support_agent/requirements.txt`:

support_agent/requirements.txt

```
google-adk
supabase
```

WHY THIS FILE

<code>requirements.txt</code>	lists Python packages that must be installed. Cloud Run reads this during build and installs them. Without it, your code imports will fail in production.
-------------------------------	---

6.4 — Deploy

PowerShell (from agents/ folder)

```
adk deploy cloud_run `
  --project YOUR_PROJECT_ID `
  --region us-centrall1 `
  .\support_agent

# when asked "Allow unauthenticated invocations?" → type: y
```

WHAT THIS COMMAND DOES

<code>adk deploy cloud_run</code>	ADK's built-in deploy command. It packages your agent, builds a container image, and deploys it to Cloud Run — you don't write a Dockerfile.
<code>--project</code>	your Google Cloud project ID (where the credits live)
<code>--region us-central1</code>	which Google data centre. us-central1 is cheapest and most available. Can be any region.
<code>.\support_agent</code>	path to your agent folder
<code>"Allow unauthenticated?"</code>	type y so DataByt can call it without a separate auth token. You'll secure it properly with a secret header later.

6.5 — Set production environment variables

Cloud Run doesn't read your local `.env` file. Set the variables directly on the service:

PowerShell

```
gcloud run services update support-agent `
  --region us-central1 `
  --set-env-vars GOOGLE_GENAI_USE_VERTEXAI=FALSE,`
GOOGLE_API_KEY=AIzaSy_your_key,`
SUPABASE_URL=https://your.supabase.co,`
SUPABASE_SERVICE_KEY=your_service_key
```

✅ CHECKPOINT 6

Deploy prints: `Service URL: https://support-agent-xxxxxxx-uc.a.run.app`

Copy that URL. Test it:

PowerShell

```
curl https://support-agent-xxxxxxx-uc.a.run.app/list-apps
```

You should see: JSON containing "support_agent". Your agent is live on the internet. 🎉

- 🔧 **Billing error** → go to `console.cloud.google.com` → Billing → link your free credits to this project.
- 🔧 **Build failed** → check `requirements.txt` exists in `support_agent/` with `google-adk` and `supabase` on separate lines.
- 🔧 **Missing env vars in production** → re-run the `gcloud run services update` command. Check there are no extra spaces in the values.

7

Wire In — DataByt calls the agent, your dashboard gets replies

~30 minutes · TypeScript, one new API route

Add the agent URL to Vercel: Settings → Environment Variables → add `SUPPORT_AGENT_URL` = your Cloud Run URL (no trailing slash).

Create this file in your DataByt repo:

```
src/app/api/support/route.ts - full file
```

```

import { NextRequest, NextResponse } from "next/server";

const AGENT_URL = process.env.SUPPORT_AGENT_URL!; // the Cloud Run URL

export async function POST(req: NextRequest) {
  // 1. Parse the request body
  let body: { message?: string; orgId?: string; ticketId?: string };
  try {
    body = await req.json();
  } catch {
    return NextResponse.json({ error: "Invalid JSON body" }, { status: 400 });
  }

  const { message, orgId, ticketId } = body;
  if (!message) {
    return NextResponse.json({ error: "message is required" }, { status: 400 });
  }

  const sessionId = ticketId ?? crypto.randomUUID(); // one session per ticket
  const userId = orgId ?? "anonymous";

  // 2. Create a session (ADK requires a session before running)
  await fetch(
    `${AGENT_URL}/apps/support_agent/users/${userId}/sessions/${sessionId}`,
    { method: "POST", headers: { "Content-Type": "application/json" }, body: "{}" }
  ).catch(() => {}); // ignore error if session already exists

  // 3. Run the agent with the user's message
  const agentResponse = await fetch(`${AGENT_URL}/run`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      app_name: "support_agent",
      user_id: userId,
      session_id: sessionId,
      new_message: { role: "user", parts: [{ text: message }] },
    }),
  });

  if (!agentResponse.ok) {
    return NextResponse.json({ error: "Agent unavailable" }, { status: 502 });
  }

  // 4. Extract the agent's reply text from the events array
  const events = await agentResponse.json();
  const lastEvent = Array.isArray(events) ? events[events.length - 1] : events;
  const replyText =
    lastEvent?.content?.parts
      ?.map((p: { text?: string }) => p.text ?? "")
      .join("") ?? "Sorry, I couldn't generate a reply.";

  return NextResponse.json({ reply: replyText });
}

```

EVERY SECTION EXPLAINED

<code>AGENT_URL const</code>	reads from env var. The ! at the end tells TypeScript "this will never be undefined" (you've set it in Vercel).
<code>Step 1: parse body</code>	reads the JSON body the dashboard sends. try/catch handles malformed JSON — returns 400 instead of crashing.
<code>sessionId / userId</code>	ADK needs these to manage conversation memory. Using ticketId as sessionId means the conversation context is preserved if the customer replies again with the same ticketId.
<code>Step 2: create session</code>	ADK requires a session to exist before you can run the agent into it. This POST creates it. .catch(() => {}) silently ignores the error if the session already exists (eg. customer is following up).
<code>Step 3: run the agent</code>	sends the message to the agent. app_name must match the agent's name="support_agent". new_message is the customer's message in ADK's format.
<code>Step 4: extract reply</code>	ADK returns an array of events. The last event's content.parts[].text is the agent's final reply. We join all parts in case the reply was split.

✔ CHECKPOINT 7 — the final test

PowerShell — test locally first

```
npm run dev
```

PowerShell — second window

```
curl -Method POST http://localhost:3000/api/support `
  -Headers @{"Content-Type" = "application/json" } `
  -Body '{"message":"How do I connect QuickBooks?","orgId":"test","ticketId":"t1"}'
```

You should see: a JSON reply with the QuickBooks steps.

Then push to production:

```
git add src/app/api/support/route.ts
git commit -m "feat: wire support agent API route"
git push
```

After Vercel deploys (~1 min), test the live site with your real URL. Your AI support agent is live end-to-end. 🎉🎉

Troubleshooting — Every Error, Exact Fix

The 4-step method (use this first, every time)

1. **Read the LAST line of the error.** Not the whole traceback — the last line names the problem.
2. **Is your venv active?** Your prompt must show `(.venv)`. 80% of "it suddenly broke" is this.
3. **Are you in the right folder?** `adk web` must run from the folder that CONTAINS your agent folders.
4. **Did you save the file?** Changes don't take effect until saved. ADK auto-reloads on save.

Error says / symptom	Root cause	Exact fix
<code>adk is not recognized</code>	venv not active	<code>.venv\Scripts\Activate.ps1</code> — prompt must show <code>(.venv)</code>
<code>ModuleNotFoundError: No module named 'google'</code>	ADK not installed in this venv	<code>pip install google-adk</code>
<code>API key not valid / 403 PERMISSION_DENIED</code>	wrong or expired Gemini key	re-copy key from <code>aistudio.google.com/apikey</code> into <code>.env</code> (no quotes, no spaces)
Dropdown empty in adk web	wrong launch folder	you're inside <code>support_agent</code> . Go up one level: <code>cd ..</code> , then <code>adk web</code>
<code>KeyError: SUPABASE_URL</code>	missing env var	add <code>SUPABASE_URL=...</code> to that agent's <code>.env</code> file
Agent invents a number (doesn't call tool)	instruction too weak, or docstring vague	add "ALWAYS use the <code>get_invoice_count</code> tool. Never guess." to the instruction. Improve the docstring.
Everything routes to one specialist	specialist descriptions too similar	make each description mention different trigger words. Check Events to see which description matched wrong.
<code>supabase.auth.error.AuthApiError</code>	using anon key instead of <code>service_role</code>	use the <code>service_role</code> key from Supabase → Project Settings → API
Cloud Run billing error	credits not linked to project	<code>console.cloud.google.com</code> → Billing → link this project
<code>502 Bad Gateway</code> from <code>/api/support</code>	<code>SUPPORT_AGENT_URL</code> wrong or missing	check Vercel env vars — no trailing slash, matches Cloud Run URL exactly
Empty reply from the agent	reply extraction failed	add <code>console.log(JSON.stringify(events))</code> , find which event has the text, adjust the <code>lastEvent</code> extraction code
<code>running scripts is disabled</code>	PowerShell execution policy	<code>Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned</code>

Reading a Python traceback

```

Traceback (most recent call last):
  File "agent.py", line 14, in get_invoice_count
    response = _supabase.table("invoices")...
  File "supabase/client.py", line 88, in table
    ...
KeyError: 'SUPABASE_URL'
```

← ignore the middle

← read THIS line first

The last line is the error. "KeyError: 'SUPABASE_URL'" tells you exactly what's wrong — that key doesn't exist in `os.environ`. The line above tells you where in your code the error occurred (`agent.py` line 14).

What's Next — How to Extend the Agent

What to add	How	Difficulty
Support chat UI in the dashboard	add a page at <code>/dashboard/support</code> that POSTs to <code>/api/support</code>	Easy
Notify yourself on escalations	in <code>escalation_agent</code> , add a tool that calls Resend or sends a Slack webhook	Medium
Real RAG over DataByt-Guide docs	embed your docs into Supabase pgvector, add a <code>search_knowledge</code> tool	Medium
Email-to-ticket via Resend inbound	add a webhook at <code>/api/webhooks/support-email</code> that creates a ticket	Medium
Agent learns from past tickets	embed resolved tickets into pgvector, retrieved by <code>search_knowledge</code>	Hard

*You now own the code — not just run it. Every line is explained, every error has a fix.
Extend it. Break it. Fix it. That's how mastery happens.*